SESSIONAL SERIES · DEPARTMENT OF CSE

Database Management System Sessional

CSE 2206 · Lecture 01 — Getting Started with PostgreSQL & SQL

Saad Ahmed
Lecturer, Dept. of CSE, BAUST

Md. Rakib Trofder
Lecturer, Dept. of CSE, BAUST

Server & Schemas · SELECT · WHERE · ORDER BY · LIMIT · INSERT · UPDATE · DELETE

WHAT WE COVER TODAY

Today's Roadmap

Six short steps — every one of them practised on the same small Library table.

1 Server → Column

How PostgreSQL organises data: host, port, databases, schemas, tables, columns.

2 The Library Table

Our working dataset for the whole session — 5 books, 6 columns.

3 SELECT

Read every column, or only the columns you actually need.

4 WHERE

Filter rows with =, > and < before anything comes back.

5 ORDER BY & LIMIT

Sort the result, then keep only the top few rows.

6 INSERT, UPDATE & DELETE

Add rows, change them, and remove them without accidents.

WHERE DOES THE DATA ACTUALLY LIVE?

From Server to Column

Before writing a single query, you must know which box you are standing in.

Server
One running PostgreSQL process

`localhost:5432`

→

Database
An isolated container of schemas

`libraryDB`

→

Schema
A namespace inside a database

`public`

→

Table
Rows of one kind of thing

`library`

→

Column
One typed field of that row

`booktitle`

Read it as one address. A server hosts many databases; a database holds many schemas; a schema holds many tables; a table is made of columns. You always connect to exactly one database at a time — to read a table in another database you must reconnect.

FIVE THINGS EVERY CLIENT ASKS FOR

Connecting to the Server

psql, pgAdmin, DBeaver, Java, Python — they all need the same five values.

Host	The machine the server runs on	`localhost`
Port	TCP port the server listens on	`5432`
User	The role you log in as	`postgres`
Password	Secret for that role	`••••••••`
Database	Which database to open	`libraryDB`

COMMAND LINE (PSQL)

```
psql -h localhost -p 5432
     -U postgres -d libraryDB
```

SAME FIVE VALUES AS A URI

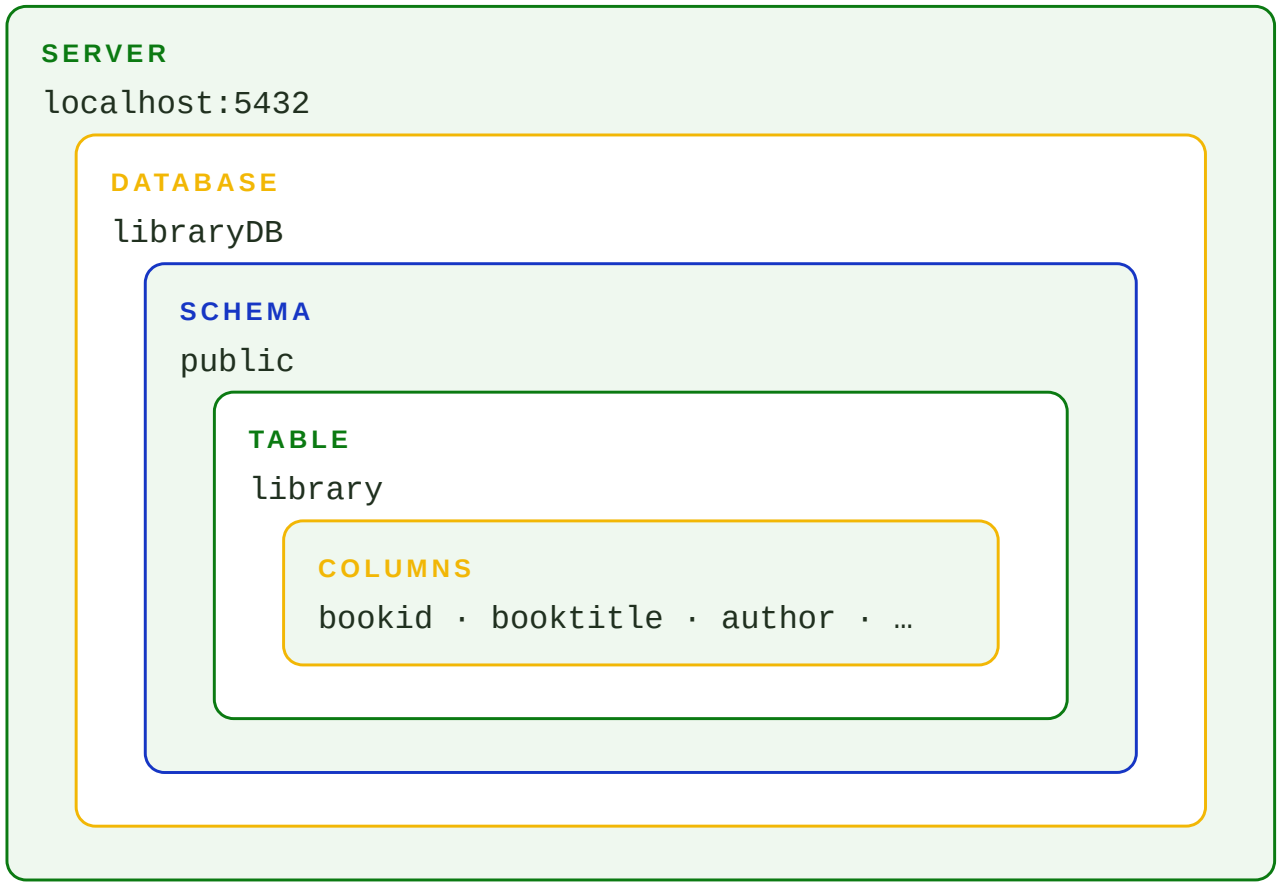
```
postgresql://postgres:secret@
        localhost:5432/libraryDB
```

If the connection fails, check the port and whether the service is actually running — before blaming your SQL.

FINDING YOUR WAY AROUND

Databases, Schemas & Tables

The same hierarchy, seen from the inside — and the commands that walk you through it.



psql SHORTCUTS

`\l`	list all databases on the server
`\c libraryDB`	connect to (switch to) a database
`\dn`	list schemas in the current database
`\dt`	list tables in the current schema
`\d library`	show the columns and types of a table
`\q`	quit psql

Unquoted names are folded to lower case — Library and library are the same table.

OUR WORKING TABLE

Defining the Library Table

Every example in this lecture runs against this one table.

DATA DEFINITION

```
CREATE TABLE Library (
    BookID          INT PRIMARY KEY,
    BookTitle       VARCHAR(120),
    Author          VARCHAR(100),
    Category        VARCHAR(50),
    Publisher       VARCHAR(100),
    AvailableCopies INT
);
```

PRIMARY KEY is the underlined attribute of your ER diagram, written in SQL. PostgreSQL will reject a duplicate or NULL BookID.

BookID

`INT · PRIMARY KEY`

Unique identity of each book — never repeats, never empty.

BookTitle / Author

`VARCHAR`

Text up to the length in brackets; quoted with single quotes.

Category / Publisher

`VARCHAR`

Repeating text values — perfect for filtering with WHERE.

AvailableCopies

`INT`

A number, so >, < and arithmetic all work on it.

FIVE ROWS, SIX COLUMNS

The Data We Start With

Load it once with INSERT, then keep these five rows in your head — you can check every answer by eye.

BookID	BookTitle	Author	Category	Publisher	AvailableCopies
1	Database System Concepts	Silberschatz	Database	McGraw Hill	6
2	Computer Networks	Andrew Tanenbaum	Networking	Pearson	5
3	Operating System Concepts	Silberschatz	Operating System	Wiley	4
4	Introduction to Algorithms	Cormen	Algorithms	MIT Press	3
5	Clean Code	Robert C. Martin	Programming	Prentice Hall	7

HOW THE ROWS GOT THERE

```
INSERT INTO Library VALUES
(1, 'Database System Concepts', 'Silberschatz',
 'Database', 'McGraw Hill', 6),
-- ... and four more rows
```

5
rows

6
columns

2
books by Silberschatz

Rows come back in no guaranteed order unless you ask for one.

READING DATA

SELECT — Fetch Everything

The star means "every column of every row". Simple, and rarely what you want in real code.

```
SELECT * FROM Library;
```

SELECT

The verb: I want to read rows.

* (star)

Every column, in the order they were defined.

FROM Library

Which table the rows come from.

Result: 5 rows × 6 columns. The semicolon ends the statement. Keywords are case-insensitive — select and SELECT both work — but writing them in capitals is the habit every reviewer expects.

READING DATA

SELECT — Only the Columns You Need

Name the columns instead of the star: less data on the wire, and you control the order.

TWO COLUMNS

```
SELECT BookTitle, Author
FROM Library;
```

YOUR ORDER, NOT THE TABLE'S

```
SELECT Author, BookTitle
FROM Library;
```

REMOVE DUPLICATES · RENAME A COLUMN

```
SELECT DISTINCT Author FROM Library;

SELECT BookTitle AS Title FROM Library;
```

DISTINCT Author returns 4 rows — Silberschatz appears once.

OUTPUT OF THE FIRST QUERY

BookTitle	Author
Database System Concepts	Silberschatz
Computer Networks	Andrew Tanenbaum
Operating System Concepts	Silberschatz
Introduction to Algorithms	Cormen
Clean Code	Robert C. Martin

Still 5 rows — choosing columns never removes rows. Only WHERE does that.

FILTERING ROWS

The WHERE Clause

WHERE tests every row and keeps only the ones where the condition is true.

Equal to

```
SELECT * FROM Library
WHERE Category = 'Database';
```

1 row

Database System Concepts

Greater than

```
SELECT * FROM Library
WHERE AvailableCopies > 4;
```

3 rows

Clean Code (7), DB Concepts (6),
Computer Networks (5)

Less than

```
SELECT * FROM Library
WHERE AvailableCopies < 5;
```

2 rows

OS Concepts (4),
Intro to Algorithms (3)

`=` equal | `<>` not equal | `>` greater | `<` less | `>=` at least | `<=` at most

Text in single quotes, and case-sensitive.

SORTING THE RESULT

ORDER BY

Without ORDER BY the server may return rows in any order it likes. Ask, and it sorts.

ASCENDING (THE DEFAULT)

```
SELECT BookTitle, AvailableCopies
FROM Library
ORDER BY AvailableCopies ASC;
```

BookTitle	AvailableCopies
Introduction to Algorithms	3
Operating System Concepts	4
Computer Networks	5
Database System Concepts	6
Clean Code	7

DESCENDING

```
SELECT BookTitle, AvailableCopies
FROM Library
ORDER BY AvailableCopies DESC;
```

BookTitle	AvailableCopies
Clean Code	7
Database System Concepts	6
Computer Networks	5
Operating System Concepts	4
Introduction to Algorithms	3

ASC is assumed if you write nothing. Text columns sort alphabetically (ORDER BY BookTitle), and you can sort by more than one column: ORDER BY Author, BookTitle. Sorting always happens after WHERE has filtered the rows.

TRIMMING THE RESULT

LIMIT — Keep Only a Few Rows

LIMIT cuts the result down after everything else has run. On its own it is unpredictable.

LIMIT alone

```
SELECT * FROM Library
LIMIT 3;
```

Returns 3 rows — but which three is up to the server. Fine for a quick peek at a big table, wrong for a report.

No ORDER BY → no promise about which rows you get.

ORDER BY + LIMIT = Top-N

```
SELECT BookTitle, AvailableCopies
FROM Library
ORDER BY AvailableCopies DESC
LIMIT 3;
```

BookTitle	AvailableCopies
Clean Code	7
Database System Concepts	6
Computer Networks	5

OFFSET 3 skips the first three rows — LIMIT 3 OFFSET 3 gives you page two.

CLAUSE ORDER MATTERS

Putting the Clauses Together

You write SELECT first — but the server does not run it first.

THE ORDER YOU WRITE

```
SELECT BookTitle, AvailableCopies
FROM Library
WHERE AvailableCopies > 3
ORDER BY AvailableCopies DESC
LIMIT 2;
```

FINAL RESULT

BookTitle	AvailableCopies
Clean Code	7
Database System Concepts	6

THE ORDER IT RUNS

1. **FROM Library** — take all 5 rows
2. **WHERE > 3** — keep 4 rows (7, 6, 5, 4)
3. **SELECT columns** — keep 2 of the 6 columns
4. **ORDER BY DESC** — sort 7, 6, 5, 4
5. **LIMIT 2** — cut to the first 2

Swap the clauses around and PostgreSQL refuses the query — this sequence is fixed.

ADDING ROWS

INSERT

One statement can add a single row or many rows at once.

NAMED COLUMNS — THE SAFE FORM

```
INSERT INTO Library
  (BookID, BookTitle, Author, Category,
   Publisher, AvailableCopies)
VALUES (6, 'Artificial Intelligence',
   'Stuart Russell', 'AI', 'Pearson', 2);
```

SEVERAL ROWS IN ONE STATEMENT

```
INSERT INTO Library VALUES
  (7, 'Compilers', 'Aho', 'Compiler',
   'Pearson', 2),
  (8, 'Software Engineering', 'Sommerville',
   'Software', 'Pearson', 4);
```

PostgreSQL replies INSERT 0 2 — the last number is the row count.

List your columns

Then the order of the table can change without breaking your INSERT.

Match value to type

Text in single quotes, numbers bare. ‘6’ and 6 are not the same thing.

Keys must be unique

Re-using BookID = 1 raises a duplicate key error — the row is rejected.

Check your work: SELECT * FROM Library;

CHANGING ROWS

UPDATE … SET … WHERE

SET says what changes; WHERE says which rows are allowed to change.

ONE ROW, ONE COLUMN

```
UPDATE Library
SET AvailableCopies = 8
WHERE BookID = 5;
```

TWO COLUMNS, AND EVERY MATCHING ROW

```
UPDATE Library
SET Publisher = 'Pearson India',
    AvailableCopies = AvailableCopies - 1
WHERE Author = 'Silberschatz';
```

→ Both Silberschatz rows change: UPDATE 2

A column can be set from its own value — AvailableCopies - 1 works per row.

UPDATE without WHERE

```
UPDATE Library
SET AvailableCopies = 0;
```

Every one of the five rows is overwritten, and there is no undo once it is committed.

BEFORE YOU RUN IT

- Run the WHERE as a SELECT first and count the rows.
- Read the reply: UPDATE 2 means two rows changed.
- Unsure? BEGIN; … ROLLBACK; puts it back.

REMOVING ROWS

DELETE

DELETE removes whole rows. There is no column list — you cannot delete "just one field".

ONE ROW, CHOSEN BY ITS KEY

```
DELETE FROM Library
WHERE BookID = 4;
```

EVERY ROW THAT MATCHES A CONDITION

```
DELETE FROM Library
WHERE AvailableCopies < 4;
```

→ **Removes Introduction to Algorithms (3 copies): DELETE 1**

Preview first: change DELETE FROM to SELECT * FROM, keep the same WHERE, and look at exactly what would disappear.

DELETE without WHERE

```
DELETE FROM Library;
```

Empties the table completely. The table itself survives — DROP TABLE is what removes the structure.

THREE WAYS TO REMOVE

`DELETE … WHERE`	chosen rows
`DELETE FROM t`	all rows, table stays
`DROP TABLE t`	rows and structure gone

HABITS THAT SAVE YOUR DATA

Three Rules for Write Queries

INSERT, UPDATE and DELETE change what is stored. These three habits cost seconds and save hours.

Preview with SELECT

- Write the WHERE clause first.
- Run it as SELECT * FROM Library WHERE …
- Only when the rows look right, swap SELECT for UPDATE or DELETE.

Wrap it in a transaction

- BEGIN; before the statement.
- Inspect the table with a SELECT.
- COMMIT; to keep it, ROLLBACK; to undo it.

Read the reply

- UPDATE 2 means two rows changed.
- DELETE 0 means your WHERE matched nothing.
- A number you did not expect is a warning.

In this lab you are working on your own copy — in a real library system, one careless DELETE removes the whole catalogue.

DO IT YOURSELF

Lab Tasks

Create the Library table, load the five rows, then answer these with a single query each.

1. Connect to libraryDB and list every table in the public schema.
2. Show BookTitle and Publisher for all books.
3. List every book published by Pearson.
4. Find the books with fewer than 5 available copies.
5. Sort all books by BookTitle from A to Z.
6. Show only the 2 books with the highest AvailableCopies.
7. Insert your own favourite book as BookID 9.
8. Add 3 copies to Clean Code, then delete the row you inserted.

Submit a single .sql file containing all eight statements, each with a one-line comment above it.

WRAP-UP

What You Can Do Now

Connect

host · port · user · password · database

Navigate

database → schema → table → column

Read

SELECT, WHERE, ORDER BY, LIMIT

Write

INSERT, UPDATE, DELETE — always with WHERE

Next session

Aggregate functions (COUNT, SUM, AVG) · GROUP BY and HAVING · joining two tables · constraints beyond PRIMARY KEY

Saad Ahmed · Md. Rakib Trofder — Department of CSE, BAUST